Testing

Introduction

Ultimately, testing is required to ensure that a developed program functions as it was designed to and meets the needs of the end user. There are various reasons why a program might not function as it should:

- Errors in the syntax mean that the program will not run properly
- Errors in the logic of the code mean that the program produces unexpected results
- Errors in the overall design of the program mean that the program doesn't do the job it was supposed to do

...and in all these cases, various testing strategies are used to ensure that these issues are eradicated, enabling the program to meet the needs of the end user.

Furthermore, the timings of these various test strategies can be crucial:

Iterative Testing

- Testing should be ongoing throughout the development process.
- You should code an aspect of your program and test it before moving on.

Final Testing

At the end of the development, when the program is complete, the program should be tested again (as a whole) against the requirements of the customer to ensure their needs have been met.

Program Errors

We will now look at the various types of errors in programs and try to understand how and when these might be detected. We will also look at errors in programs and state how it might be corrected. Finally we will look at the different testing strategies.

There are three type of errors which may occur in a program. They are:

- **Syntax**
- **Logic**
- **Run-Time Errors**

We shall now look at these in turn.

Syntax Errors

Remember that the source code we write cannot be understood by a CPU in its current form. For it to be understood it must be translated into binary code and for a translator to be able to do this correctly, code must be written according to the rules of the language. A syntax error is simply an error where the code written doesn't meet the rules of the programming language. These errors appear when the source code is translated into

Computer Science UK Membership Site Licence: Do not share outside your centre.

machine code. The translator tries to convert the code but if the code doesn't meet the rules known to the translator, it throws up an error.

Syntax Errors

Examples of syntax errors:

- Incorrect assignment statement. E.G.:

number 1 + number 2 = answer

- Missing/extra punctuation or brackets. E.G.:

print("hi)

- Not enough arguments for functions. E.G.:

```
def printer(a, b):
    print(a+b)

a, b= 2, 3
printer(a)
```

- Missing part of a multi-line statement. E.G.:

Stating an IF statement but failing to finish with an END-IF (in some languages)

- Misspelling an identifier after its already been declared. E.G.:

number1 = 5

answer = numbr1 + number2

- Type mismatch – when the computer expects a certain data type but receives another. E.G.:

number = int(input("Enter a number")) ...but the user enters the word 'two'

Some IDEs have translator diagnostics and highlight potential syntax errors. Some may even correct them. But, these can be erroneous in themselves and so the programmer should always check first.

Logic Errors

A logic error is one where the code is written in accordance with the programming rules and is therefore translated and runs, however, the program produces unexpected results.

A common logic error is an incorrect maths statement. For example, if the programmer wrote **time/distance** when calculating speed, the program would output a value, but it would be incorrect. This is because **speed = distance/time**.

Computer Science UK Membership Site Licence: Do not share outside your centre.

Logic errors can be harder to spot as programs will compile and run. The programmer needs to test against expected results to ensure that the program is free of logical errors.

Other examples of logic errors are:

- Wrongly ordered instructions.
- Wrong conditions for IF statements or iterations.

Run-Time Errors

When the code is free of syntax errors and logic errors, there is still one more error that can result. A run-time error will occur in a normal working program if some extreme conditions occur. One example maybe that the program has got into a situation where it is to perform arithmetic which has an impossible answer:

- Dividing a number by zero
- Finding the square root of a negative number

These are known as **arithmetic errors**.

Other common run-time errors are:

Overflow Error – This is when a variable tries to hold a value which is too big for it.

An integer holds 2 bytes (-32768 to 32767), so if a larger value is calculated and assigned into this variable, we will get an overflow error.

A solution may be to round the answer and temporarily remove the zeros (less precise but the most important information about the number can be stored). Zeros can be added when answer is to be displayed or calculation continued.

Stack Overflow Error – This is when stack space runs out – often occurs during recursion.

Library Error – This may occur if your program hasn't importuned the correct library, or if the program tries to access a library that is not present.

Testing Strategies

Black Box Testing

This only deals with the inputs and outputs of the program and not how the algorithms work. Ideally we would test that EVERY possible input produces the expected results – but this is not possible due to the great combinations of possible inputs. So similar inputs are grouped. In Black Box testing we use 'Valid', 'Invalid/Erroneous' and 'Borderline/Extreme' data.

White Box Testing

This only deals with the algorithms to make sure that they function correctly. The focus is on testing that all possible paths of the algorithms work as they should. On each test, the path of the execution is noted and compared with other runs. Each path is determined by the values of the conditions in constructs such as IFs or Loops.

Computer Science UK Membership Site Licence: Do not share outside your centre.

Test Plans

These are used to:

- Ensure that a system is fully tested
- To document the outcomes.

Test plans are made up of:

- The test data
- The reason for the test
- The expected outcome
- The actual result (which is added later – after development)

Valid data – should produce the expected results.
Invalid data – should produce an error.
Borderline – it is important to test that data on the edge between valid and invalid are dealt with correctly by the program.

Ultimately they will test the program works as it should – and has met the success criteria.

So if the success criteria was:

Example Success Criteria:
1. User should be able to enter ONLY dice number 4, 6 or 12
2. Program should generate a random number within the limits of the dice chosen.
3. The program should ask if the user would like to roll again.

A suitable test plan might look something like this:

Test Number	Reason for Test	Type of Test	Test Data	Expected Outcome
1	To see if the user can input their dice number	Valid	Enter a 6 when program requests	The program will accept the input and pick a random number from 1-6.
2		Borderline	Enter a 12 when program requests	The program will accept the input and pick a random number from 1-12.
3		Invalid	Enter a 1 when program requests	The program will NOT accept the input and ask for the number again.
4	To see if the user can roll again.	Valid	Type in "Y"	The program will ask the user for a dice number again.
5		Valid	Type in "N"	The program close.
6		Invalid	Type in "Yes"	The program will ask the user if they would like to roll again.

Dry Runs and Trace Tables

At times, when errors are not as obvious to spot, it can be helpful to go through the code manually. Executing the code (in your head) and recording the effects on the various variables. This is a 'Dry Run'. A dry run is often done using a trace table. A trace table will have a column for the line of code to be executed, a column for each variable affected and a column for any values outputted. Sometimes adding a column for comments is useful.

Computer Science UK Membership Site Licence: Do not share outside your centre.

```
def mystery(n):
    a, b = 0, 1
    while a < n:
        print (a)
        a, b = b, a + b
```

If n = 40

Iteration	a	b	Output	Comment
1	0	1	0	
2	1	1	1	
3	1	2	1	
4	2	3	2	
5	3	5	3	
6	5	8	5	
7	8	13	8	
8	13	21	13	
9	21	34	21	
10	34	55	34	Loop ends, as on next iteration, a will be greater than 40

Acceptance Testing

The previous tests have all been carried out to ensure that there are no bugs in the software. Acceptance testing takes place to ensure that the final product (now free of errors), meets the needs of the user and client. Here, the program is tested against the requirements set out by the developers and end users before the development process began.

Alpha and Beta testing are examples of this...

Alpha Testing

After the product has been 'Black/White Box Tested' it is tried by the programmers, who pretend to be the end users (in-house), who check to see if the program has met its objectives and whether it does what it should do. Here the major flaws of the program can be quickly dealt with by the programmers themselves.

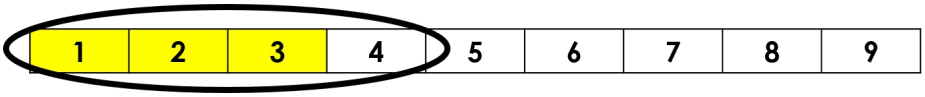
Beta Testing

After a program has been Alpha tested it is beta tested. This is the last test before it is widely distributed. It often involves large numbers of 'real' users, using the software under realistic conditions – often heavy duty demanding users. It is the last chance to find any bugs or short-comings.

Questions *(The question zone you choose must either match your target grade or be higher!)*

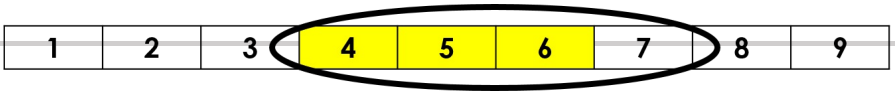
Computer Science UK Membership Site Licence: Do not share outside your centre.

Question Zone 1-3



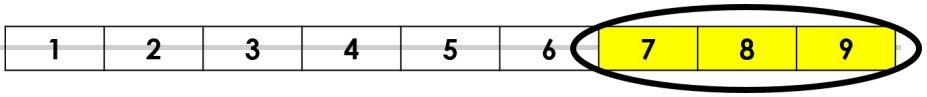
1. Provide three reasons why programs need to be tested. [3]
2. What is meant by the term 'Iterative Testing'?[3]
3. Describe the differences between 'Syntax', 'Logic' and 'Run-Time' errors. Provide examples to support your answer. [6]

Question Zone 4-6



1. Describe the differences between 'Syntax', 'Logic' and 'Run-Time' errors. Provide examples to support your answer. [6]
2. Describe the differences between 'Black Box' and 'White Box' testing. [4]
3. Explain what is meant by the term 'Acceptance Testing', providing examples of the different stages of 'Acceptance Testing'.[5]

Questions Zone 7-9



1. Explain what is meant by the term 'Acceptance Testing', providing examples of the different stages of 'Acceptance Testing'.[5]
2. Explain the differences between 'Black Box' and 'White Box' testing, describing examples of testing methods used for each form of testing. [6]
3. Discuss the need for 'normal', 'extreme' and 'erroneous' test data, when conducting 'Black Box' testing.[4]

Computer Science UK Membership Site Licence: Do not share outside your centre.

Checklist:

- Date and title, clearly presented
- Spelling & grammar checked
- Question numbers in the margin
- Handwriting neat & legible
- Punctuation / Capital letters

Keywords / Key Terms:

Iterative Testing: Testing that is ongoing throughout the development process / whilst the code is being written.

Black Box Testing: A method of testing that only deals with the inputs and outputs of the program and not how the algorithms work.

White Box Testing: A method of testing that only deals with the algorithms to make sure that they function correctly.

Computer Science UK Membership Site Licence: Do not share outside your centre.

Stick answer sheet here

State/Identify/Give/Name: Simply label a diagram, fill out a table or write a few words
Describe: Describing is 'saying what you see' (E.G.: A computer will have a CPU, Primary and Secondary storage etc)
Explain: Explaining is 'saying WHY/HOW something is like that'. (E.G.: A computer will have a CPU so that it can process all of the data the computer needs to perform a range of tasks. Primary and Secondary storage is needed because...)
Discuss: Discussing is 'looking at two sides of an issue, weighing up the two views and giving a conclusion'. Often these require a mini essay answer. (E.G.: New technology could be seen as being bad for the environment because..., but on the other hand, new technology has led to... In conclusion I believe that...)
Describe/Explain/Discuss **using examples**: Finally, if you are asked to give examples in any of these types of questions – YOU MUST **GIVE EXAMPLES**!

Computer Science UK Membership Site Licence: Do not share outside your centre.

Reflections **Score:** / **Percentage:** % **Grade:** **Progress** On / Above / Below

What Went

- ☐ I demonstrated a good level of understanding.
- ☐ I responded to the command words effectively.
- ☐ My answers were detailed / were written in depth.
- ☐ My work was well presented / legible.
- ☐ My answers effectively incorporated technical terminology.
- ☐ My responses were well structured / organised.
- ☐ My revision strategy was effective as I showed depth of understanding in my answers.
- ☐ My answers contained enough points / examples / explanations to achieve the marks available.

Even Better

- ☐ My answers need to be more accurate.
- ☐ I must respond correctly to the command words.
- ☐ My answers need more detail / greater depth.
- ☐ I must take greater care over my work / write neatly.
- ☐ I must incorporate key terminology into my answers.
- ☐ I must better organise my answers to improve its clarity.
- ☐ I need to improve my revision strategy, as I did not demonstrate a depth of understanding in my answers.
- ☐ My answers didn't contain enough points / examples / explanations to achieve the marks available.

Further thoughts:

Computer Science UK Membership Site Licence: Do not share outside your centre.